

# **SPM: Workload Balancing**

---

Academic year 2025-2026

Prof. Massimo Torquati  
[massimo.torquati@unipi.it](mailto:massimo.torquati@unipi.it)

# Outline

---

- Data partitioning
- Static data distribution policies (block, cyclic, and block-cyclic)
- Irregular workloads and dynamic assignment of tasks

# Data Partitioning

---

- Data Parallelism is a programming model where the same operation (function) is applied in parallel to multiple independent *partitions* of a data set
- The parallelism is exploited through a set of Workers, each computing the elements of the partition
- Distinct partitions of the dataset can be computed independently by Workers (*map paradigm*) or, instead their computations requires interactions (i.e., message exchanges or synchronizations) among Workers (*stencil paradigm*)
- **Data partitioning** is the strategy of dividing input data among available threads or processes
  - Data partitioning directly influences the efficiency and performance of data parallel computations
- A key challenge arises when Worker's workload is not evenly distributed (**workload imbalance**)
  - Some threads/processes will finish later, causing other threads/processors to wait idle
- Effective workload balancing techniques are crucial to ensure balanced workload distribution, optimal resource use, and overall performance improvement

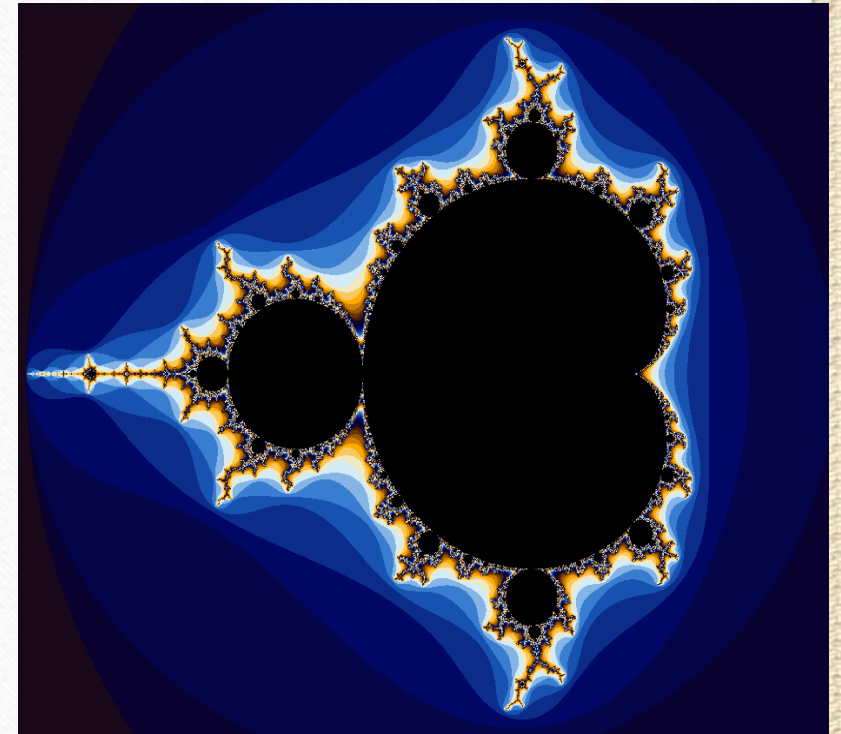
# Example: Mandelbrot set

1/2

- The Mandelbrot set is the set of all complex numbers  $c \in \mathbb{C}$  for which the sequence defined by the iteration  $z_n(c)$  remains bounded as  $n \rightarrow \infty$

$$z_n(c) = \begin{cases} z_0 = 0 \\ z_{n+1} = z_n^2 + c \end{cases}$$

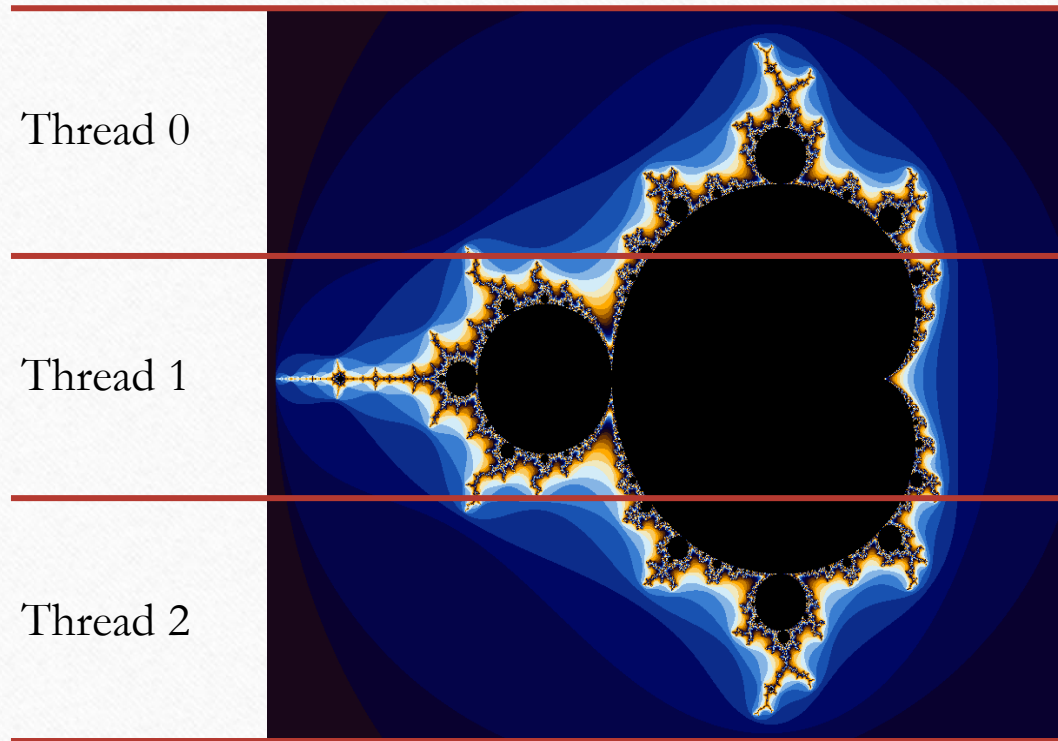
- In practice, it is checked whether  $|z_n(c)| \leq 2$  for some large number of iterations (e.g., 1000). If this holds, it is assumed that  $c$  is part of the Mandelbrot set
- To obtain a nice fractal, the color of each pixel depends on the number of iterations required until the magnitude of  $z_n(c)$  exceeds 2
  - The color of each pixel of the figure can be computed independently (*embarrassingly data parallel computation*)
- Black pixels (i.e., points belonging to the Mandelbrot set) require the maximum number of iterations, while the colored pixels outside the set usually escape in fewer iterations
  - The per-pixel computational load is highly skewed
  - Points near the boundary can also take many iterations



# Example: Mandelbrot set

2/2

- What would happen if we computed the Mandelbrot set using 3 threads, partitioning the pixels as shown in the figure?



Thread 1 performs most of the work.  
The threads' workload is unbalanced,  
leading to low overall efficiency

# Data Distribution Policies

1/2

- Key question: How can we aggregate data into tasks and assign them to the available processing elements (Workers) so as to ensure balanced computation?
- In general, the assignment of tasks can be done *statically* or *dynamically*
- Static task assignment
  - Input data collection is partitioned once at the beginning, before the computation/kernel starts
  - Each Worker receives a fixed partition and processes it independently
  - The final output is partitioned accordingly (output partitions might be gathered or reduced into an output collection)
  - Typically, each partition has roughly the same size (or better yet, a similar computational cost)
- Dynamic task assignment
  - Input data collection is split into small partitions (**chunks**)
  - Chunks are assigned to Workers **on-demand** using a dedicated scheduler, or they pull tasks from a central queue (**work sharing**) or steal work from other Workers (**work stealing**)
  - **Small chunks produce better load balance but higher scheduling overhead and weaker locality**

# Data Distribution Policies

2/2

- **Static policies**
  - The focus is to identify a data partitioning that evenly distribute the workload
  - The ideal use case has a regular workload
  - No dynamic task scheduling overhead
  - Standard policies: **block**, **cyclic**, **block-cyclic**
  - Example: Dense Matrix-Vector product, Regular Stencil-based Computation, Convolution, etc.
- **Dynamic policies**
  - Adapt tasks assignment to Workers to handle irregular workloads
  - Improved efficiency for workloads with skewed computational demands
  - Additional overhead due to tasks scheduling (i.e., extra messages/synchronizations for coordination)
  - Standard policies: **on-demand**, **work-sharing** and **work-stealing**
  - Example: Mandelbrot set, graph-based algorithms, N-Body Simulations, Sparse Matrix-Vector product, ...

# Matrix-Vector Multiplication Example

$$A \in \mathbb{R}^{m \times n} \text{ and } x \in \mathbb{R}^n \quad b_i = \sum_{j=0}^{n-1} A_{ij} \cdot x_j \quad \text{for all } i \in \{0, \dots, m-1\}$$

- The minimal task granularity is the computation of one  $b_i$ . We need  $A_{i*}$  (i.e.,  $i$ -th row of  $A$ ) and  $x_*$  (i.e., all  $x$ ).
- Each  $b_i$  (i.e., each task) can be computed independently (dot-product)
- In real-world scenarios,  $m$  and  $n$  are large numbers (i.e.,  $\gg$  #cores of the machine), therefore to maximize single processor utilization, **each processor computes a partition of  $k$  tasks**

This is a regular workload, so static partitioning is usually sufficient

# Static Data Distribution Policies

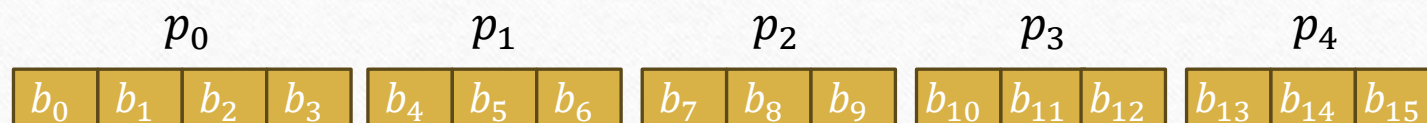
1/3

Example  $p=5$   $m=16$



- Block task distribution:**

The block size is at least  $\left\lceil \frac{m}{p} \right\rceil$ .



If  $(m \bmod p) = k \neq 0$  then  $\forall p_i \mid i < k$  the block size is  $\left\lceil \frac{m}{p} \right\rceil$

```
auto block = [&] (const index_t& id) -> void {  
    // compute chunk size, lower and upper task id  
    const index_t chunk = SDIV(m, num_threads);  
    const index_t lower = id*chunk;  
    const index_t upper = std::min(lower+chunk, m);  
  
    // only computes rows between lower and upper  
    for (index_t row = lower; row < upper; row++) {  
        value_t accum = value_t(0);  
        for (index_t col = 0; col < n; col++)  
            accum += A[row*n+col]*x[col];  
        b[row] = accum;  
    }  
};
```

$$\text{SDIV}(x, y) = \left\lceil \frac{x + y - 1}{y} \right\rceil \geq \frac{x}{y} \quad \text{for all } x, y \in \mathbb{R}^+$$

For simplicity, this version uses a ceil-based block size. SDIV-based block partitioning ensures full coverage but may leave some threads idle when  $m$  is not divisible by  $p$ . A more balanced implementation distributes the remainder explicitly across the first threads

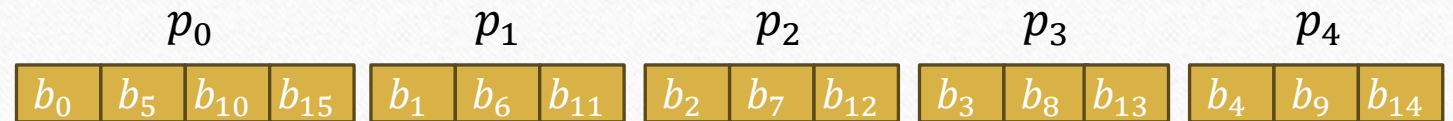
# Static Data Distribution Policies

2/3

Example  $p=5$   $m=16$ 

|       |       |       |       |       |       |       |       |       |       |          |          |          |          |          |          |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|
| $b_0$ | $b_1$ | $b_2$ | $b_3$ | $b_4$ | $b_5$ | $b_6$ | $b_7$ | $b_8$ | $b_9$ | $b_{10}$ | $b_{11}$ | $b_{12}$ | $b_{13}$ | $b_{14}$ | $b_{15}$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|

- **Cyclic task distribution:**



- The task  $t_i$  is assigned to  $p_{i \bmod p}$

```
auto cyclic = [&] (const index_t& id) -> void {  
    // indices are incremented with a stride of p  
    for (index_t row = id; row < m; row += num_threads) {  
        value_t accum = value_t(0);  
        for (index_t col = 0; col < n; col++)  
            accum += A[row*n+col]*x[col];  
        b[row] = accum;  
    }  
};
```

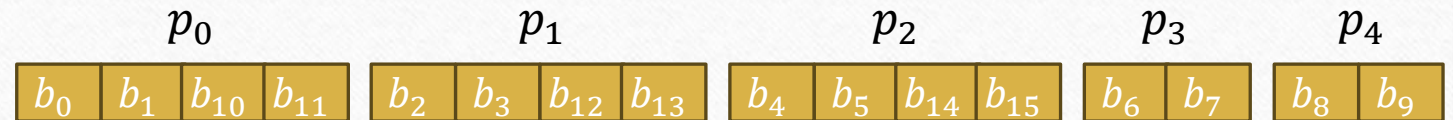
# Static Data Distribution Policies

3/3

Example  $p=5$   $m=16$



- Block-Cyclic task distribution:



Given a block of size  $c > 0$  ( $p \cdot c$  is called *stride*)  
then  $t_i$  is assigned to processor  $p_{(i \div c) \bmod p}$

The cyclic distribution has  $c = 1$

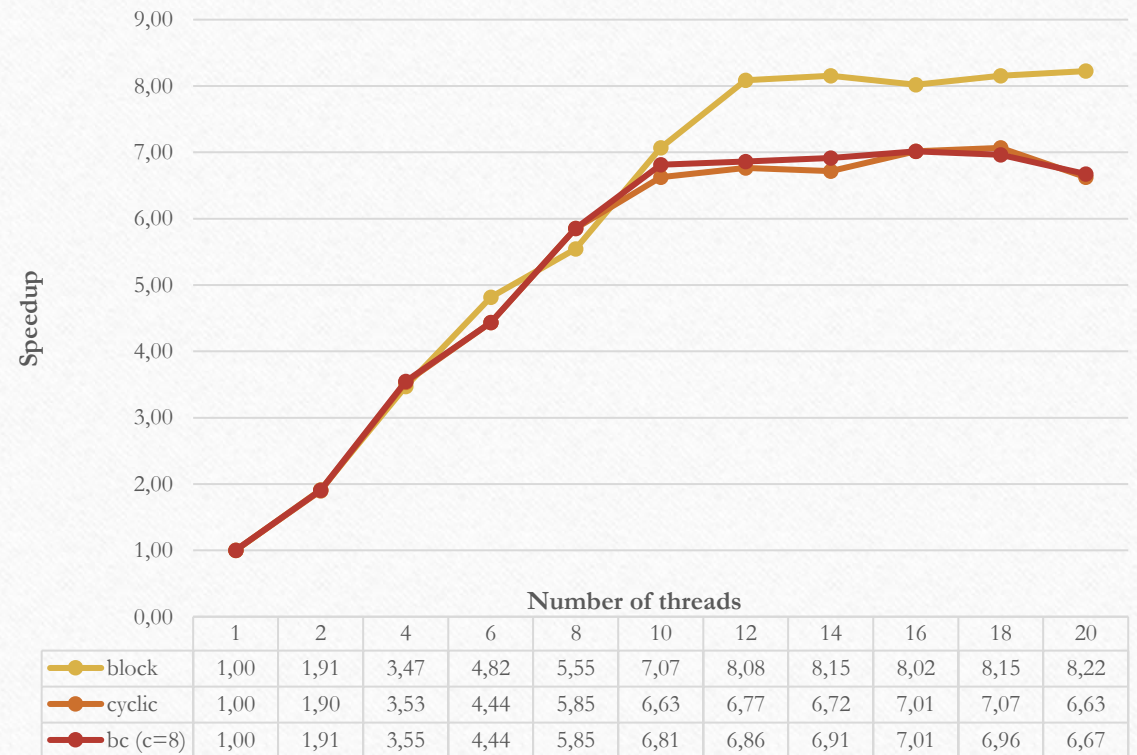
```
auto block_cyclic = [&] (const index_t& id) -> void {  
    // precompute the stride  
    const index_t stride = num_threads*chunk_size;  
    const index_t offset = id*chunk_size;  
  
    // for each block of size chunk_size in cyclic order  
    for (index_t lower = offset; lower < m; lower += stride) {  
        // compute the upper border of the block  
        const index_t upper = std::min(lower+chunk_size, m);  
        // for each row in the block  
        for (index_t row = lower; row < upper; row++) {  
            value_t accum = value_t(0);  
            for (index_t col = 0; col < n; col++)  
                accum += A[row*n+col]*x[col];  
            b[row] = accum;  
        }  
    }  
};
```

# Testing different static policies

- Test on the front-end node of the spmcluster:  
10 repetitions for each point, min and max removed and then computed the average (variance not reported)
- $m = 2^{18}$ ,  $n = 2^{15}$ , sequential version takes about 9.54s
- The block-cyclic distribution (bc) uses  $c=8$  as the chunk size.
- $E(20) = \frac{8.22}{20} \cong 41\%$
- Why is the Efficiency low?
  - $I = \frac{2 \cdot m \cdot n}{8 \cdot (m \cdot n + n + m)} \cong 0.25 \text{ FLOPS/byte}$
  - Low arithmetic intensity; the algorithm is memory bound

In this regular, memory-bound kernel, better locality matters more than finer-grain balancing

Matrix Vector Multiplication  $m=18$ ,  $n=15$



# What about irregular data structures?

- Some data structures are irregular
- A Triangulated Irregular Network (TIN) consists of a collection of non-overlapping triangles with highly variable sizes/shapes/density and is inherently irregular
  - Regions with steep terrain need many small triangles; flat regions need few large ones
- Straightforward static partitioning may lead to severe workload imbalance between Workers
  - Graph/Geometry-based partitioning can help distribute the workload more evenly (domain specific algorithms)
- Even with good static partitioning policies, the active workload may **change over time**
- In many such cases, dynamic task distribution policies are needed to adapt to changing workloads

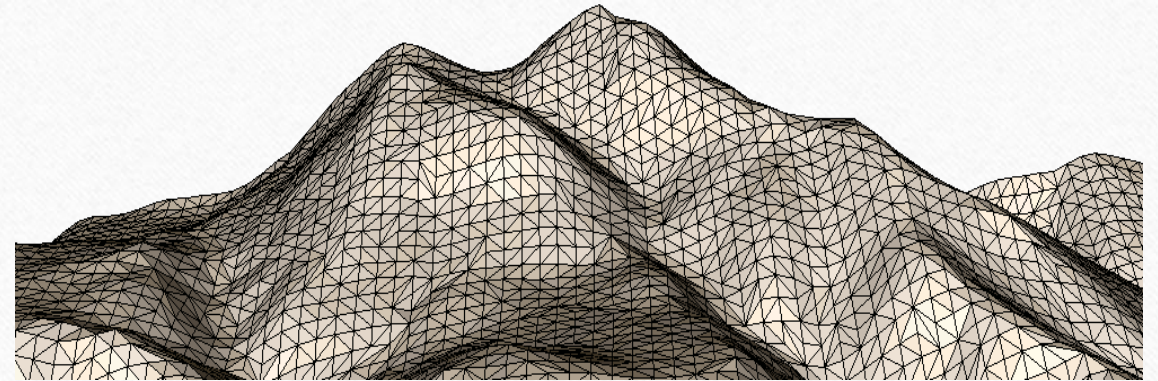


Image from: <https://www.geom.at/terrain-triangulation/>

# Unbalanced workloads

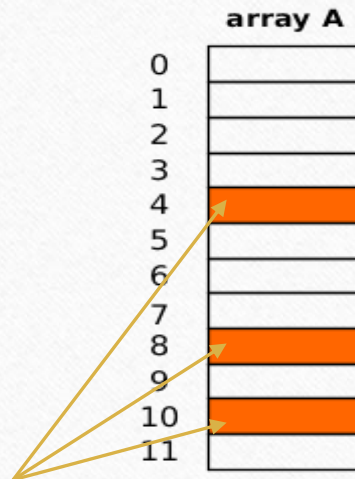
---

- In the static data distribution, the data assignment is predetermined at the program start
  - No extra overhead at runtime for assigning tasks to threads
- Static approaches work well if the computation per data element does not vary too much and we know in advance the number of tasks to compute
  - The computation of the Mandelbrot set is an example of unbalanced workload
- One option is to use very small chunk sizes of tasks and block-cyclic assignment strategies
  - However, this strategy does not work well in all cases. When tasks are heavily skewed, only a few more tasks per thread may produce load imbalance
    - In the worst-case scenario, all heavy tasks can be assigned to the same thread
  - Additionally, static assignments cannot be used when the number of tasks is not known statically
- For unbalanced workloads, it is better to use dynamic assignment strategies

# Arrays with non-uniform per-element cost

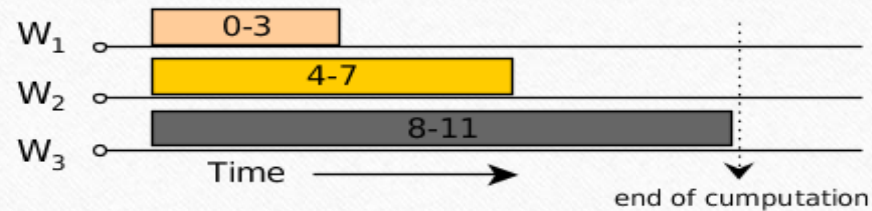
Suppose to have 3 workers and a chunksize=2, then the initial plan used for scheduling iterations is

| wid | #tasks | min-max |
|-----|--------|---------|
| 0   | 2      | 0-3     |
| 1   | 2      | 4-7     |
| 2   | 2      | 8-11    |

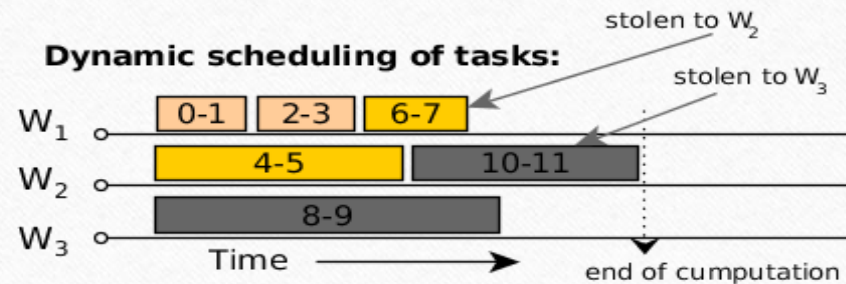


Compute-intensive  
array entry

**Static assignment of tasks:**



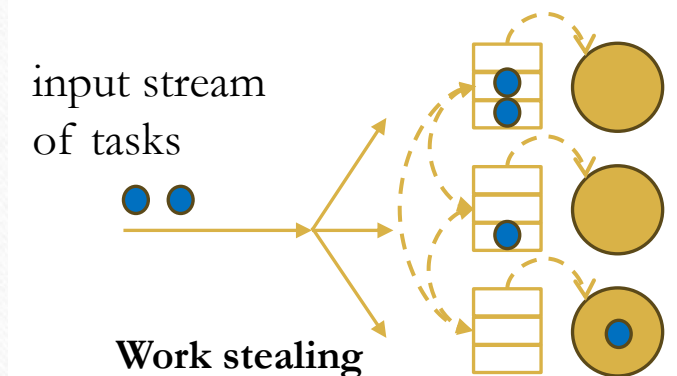
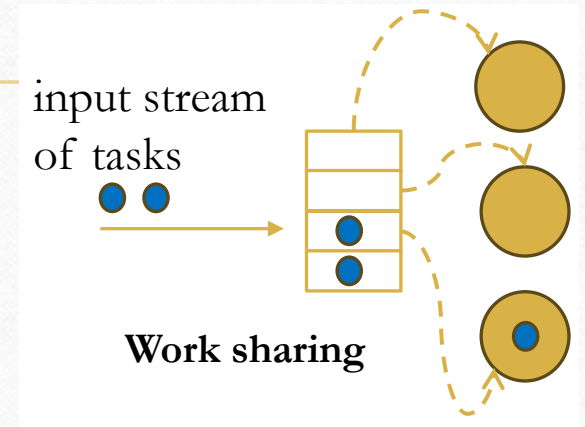
**Dynamic scheduling of tasks:**



Work-sharing and work-stealing  
can reassign chunks when  
Workers become idle.  
This improves load balance

# Work-Sharing and Work-Stealing task scheduling

- **Work-Sharing:** all Worker threads share **one** concurrent task queue
  - The queue is MPMC and can be implemented as lock-based or lock-free (often bounded)
  - The threadPool is one possible implementation of the Work-Sharing pattern
  - Simple and works well with **few producers & medium/large tasks**; may suffer **contention** on the queue.
- **Work-Stealing:** each Worker thread maintains **a local task queue**; tasks may be assigned to Workers using a round-robin or random policy, or, instead, start on a Worker; idle Workers (i.e., those with empty local queue) randomly steal (a batch of) tasks from other Workers' queue
  - Workers push/pop at the bottom (queue tail), thieves steal from the top (head)
  - Improved locality and less contention
  - Best for **many tiny irregular tasks**; **less contention** and occasional steal costs.

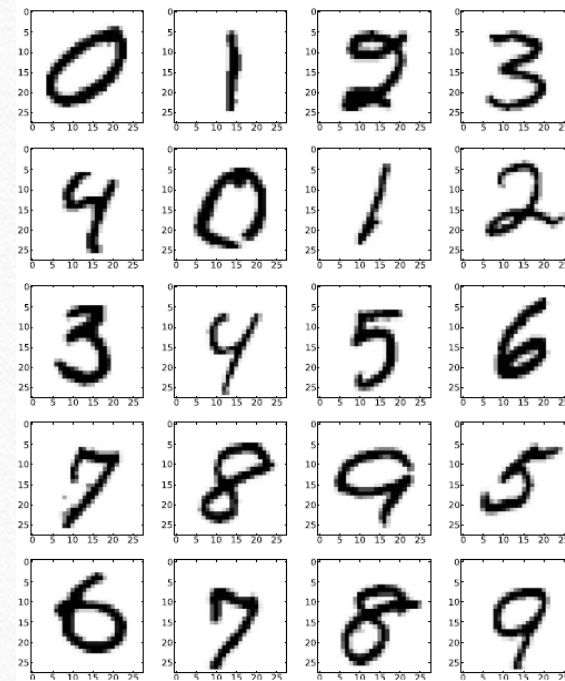


# Example: All-pairs distance Matrix

- We have a matrix  $D_{ij}$  of shape  $m \times n$  where  $i$  denotes one of the  $m$  vectors (for example, a vector is all pixels of a given image) and  $j$  enumerates all  $n$  elements of the vectors.
- We want to compute the distance (or more generally, the similarity)  $d(\cdot, \cdot)$  between all pairs in  $D$ 
  - $\Delta_{ij} = d(x^{(i)}, x^{(i')})$  for all  $i, i' \in \{0, \dots, m - 1\}$
- The distance/similarity measure  $d(\cdot, \cdot)$  might be a traditional metric such as Euclidean distance or any symmetric binary function that assign a notion of similarity to pair of instances
- We have to calculate  $m^2$  distance/similarity scores between vectors. Assuming that the computation of a single score value takes  $O(n)$ , we have  $O(m^2n)$  operations to compute the distance matrix  $\Delta$
- However, the matrix  $\Delta$  is symmetric  $\Delta_{ij} = d(x^{(i)}, x^{(i')}) = d(x^{(i')}, x^{(i)})$  for all  $i, i' \in \{0, \dots, m - 1\}$ , thus we have to calculate the lower triangular part of the matrix  $\Delta$  and then copy the elements in the corresponding position in the upper triangular part.

# Example: All-pairs distance Matrix

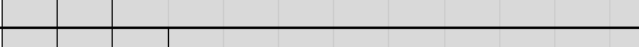
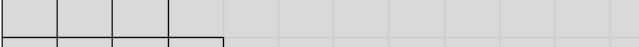
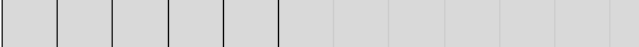
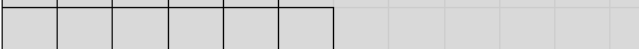
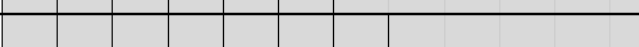
- Let us consider as  $D$  the MNIST dataset. It consists of 70,000 handwritten digits stored as gray-scale images of shape  $28 \times 28$ .
  - 60,000 images coming from the training dataset and 10,000 coming from the test dataset
- For our purpose, we interpret each of the  $m=70,000$  images as plain vectors each with  $n=784$  points. All  $m$  images are stored in the matrix  $D_{ij}$ 
  - For MNIST, storing the full 70,000 x 70,000 distance matrix requires about 19.6 GB in single precision, so memory footprint is already a major issue.
- Additionally, for the sake of simplicity,  $d(\cdot, \cdot)$  is the Euclidean distance
  - $$d(x^{(i)}, x^{(i')}) = \|x^{(i)} - x^{(i')}\|^2 = \sum_{j=0}^{n-1} (D_{ij} - D_{i'j})^2$$
- Due to the symmetry of  $\Delta_{ij}$  we only have to compute  $i+1$  entries in row  $i$ 
  - In  $\Delta_{ij}$  we have the distance between the  $i$ -th element and the  $j$ -th element, and  $\Delta_{ij} = \Delta_{ji}$



# Example: All-pairs distance Matrix

- A naive parallelization of the problem uses a static block-cyclic distribution of row elements of the matrix  $\Delta$  to threads for some value of the chunk  $c$ .

For example, considering a  $12 \times 12$  all-pairs distance matrix and  $c=2$ , we have

|                     |                                                                                    |    |
|---------------------|------------------------------------------------------------------------------------|----|
| thread <sub>0</sub> |   | 3  |
| thread <sub>1</sub> |   | 7  |
| thread <sub>2</sub> |   | 11 |
| thread <sub>0</sub> |   | 15 |
| thread <sub>1</sub> |   | 19 |
| thread <sub>2</sub> |  | 23 |

- The workload of the threads is *unbalanced*
- It can be better with  $c=1$  but still it remains unbalanced
  - $\text{thread}_0$  22 tasks,  $\text{thread}_1$  26 tasks,  $\text{thread}_2$  30 tasks
- Row  $i$  computes  $i+1$  distances, therefore its work is  $O((i+1) * n)$
- We need a different strategy that assigns elements to compute to threads dynamically

# All-pairs distance matrix: Static vs Dynamic assignment

```
auto block_cyclic = [&] (const index_t& id) -> void {
    // precompute offset and stride
    const index_t off = id*chunk_size;
    const index_t str = num_threads*chunk_size;
    // for each block of size chunk_size in cyclic order
    for (index_t lower = off; lower < rows; lower += str) {
        // compute the upper border of the block (exclusive)
        const index_t upper = std::min(lower+chunk_size, rows);
        // for all entries below the diagonal (i'=I)
        for (index_t i = lower; i < upper; i++) {
            for (index_t I = 0; I <= i; I++) {
                // compute squared Euclidean distance
                value_t accum = value_t(0);
                for (index_t j = 0; j < cols; j++) {
                    value_t residue = mnist[i*cols+j]
                                    - mnist[I*cols+j];
                    accum += residue * residue;
                }
                // write Delta[i,i'] = Delta[i',i]
                all_pair[i*rows+I] =
                all_pair[I*rows+i] = accum;
            }
        }
    }
};
```

Rows assigned statically  
using offset + stride

Next chunk acquired at  
runtime from a shared counter

Static block-cyclic

```
auto dynamic_block_cyclic = [&] (const index_t& id) -> void {
    index_t lower = 0;
    // while there are still rows to compute
    while (lower < rows) {
        // update lower row with global lower row
        {
            std::lock_guard<std::mutex> lock_guard(mutex);
            lower = global_lower;
            global_lower += chunk_size;
        }
        // compute the upper border of the block (exclusive)
        const index_t upper = std::min(lower+chunk_size, rows);
        // for all entries below the diagonal (i'=I)
        for (index_t i = lower; i < upper; i++) {
            for (index_t I = 0; I <= i; I++) {
                // compute squared Euclidean distance
                value_t accum = value_t(0);
                for (index_t j = 0; j < cols; j++) {
                    value_t residue = mnist[i*cols+j]
                                    - mnist[I*cols+j];
                    accum += residue * residue;
                }
                // write Delta[i,i'] = Delta[i',i]
                all_pair[i*rows+I] =
                all_pair[I*rows+i] = accum;
            }
        }
        lower = upper;
    }
};
```

Dynamic block assignment

Only the assignment policy changes. The kernel is the same

# Example: All-pairs distance Matrix

- Results obtained running the *all\_pair.cpp* implementation for the MNIST dataset on the cluster front-end node.
- Block-cyclic and Dynamic assignments varying  $c$  and using 40 threads in total
- Static block-cyclic reduces imbalance, but cannot eliminate it because row cost grows with the row index
- Dynamic assignment improves balance, at the price of synchronization overhead

| Static block-cyclic distribution |      |      |      |
|----------------------------------|------|------|------|
| chunk size (c)                   | 1    | 8    | 64   |
| Speedup(40)                      | 34.9 | 34.4 | 30.3 |

| Dynamic distribution |      |      |      |
|----------------------|------|------|------|
| chunk size (c)       | 1    | 8    | 64   |
| Speedup(40)          | 37.3 | 36.5 | 31.8 |

# Computing the Mandelbrot set

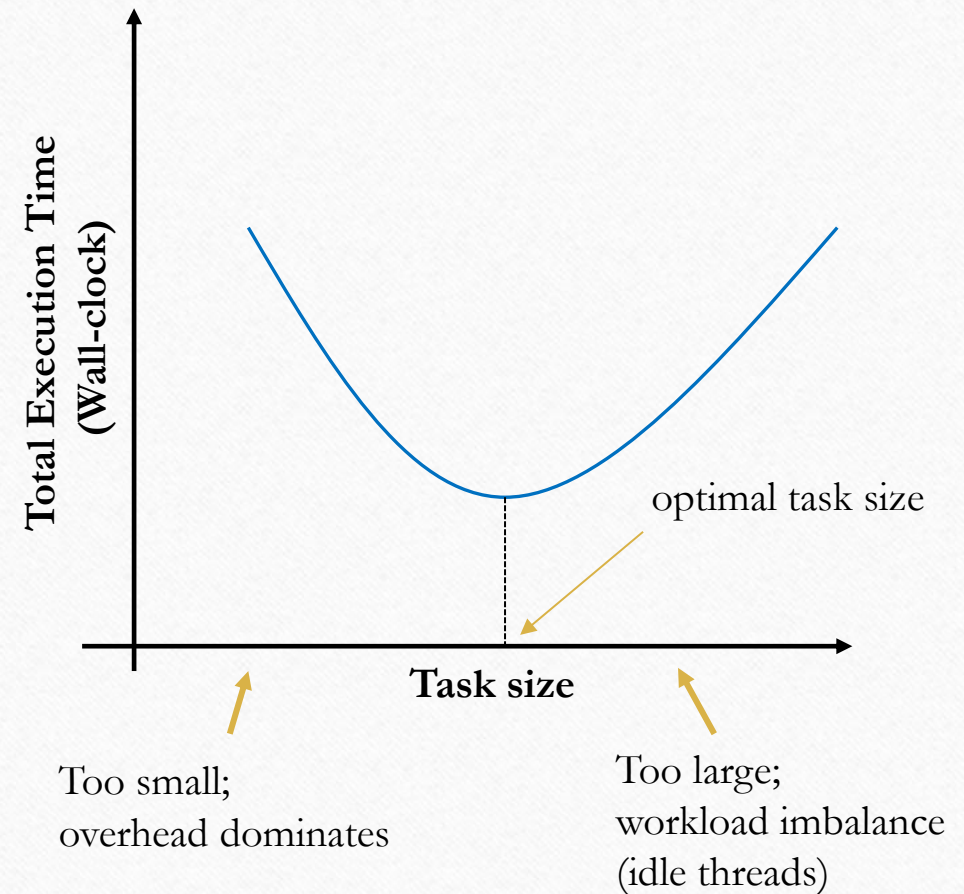
- Results obtained running the *mandel\_\*.cpp* implementation for the Mandelbrot set problem on a 64 core machine (2 AMD EPYC 7551 2-way SMT - 128 logical cores @ 2GHz)
  - All code compiled with `-DNO_DISPLAY`
  - `-O3 -march=native`
- Sequential time: 146.9
- Block-cyclic and dynamic distribution varying  $c$  and using 128 threads
- For this irregular workload, dynamic scheduling improves balance, while the threadPool-based work-sharing solution adds only moderate overhead

| Static block-cyclic distribution |      |      |     |      |
|----------------------------------|------|------|-----|------|
| chunk size (c)                   | 1    | 4    | 8   | 16   |
| Block-Cyclic                     | 1.96 | 2.38 | 3.6 | 6.94 |

| Dynamic distribution |      |      |      |      |
|----------------------|------|------|------|------|
| chunk size (c)       | 1    | 4    | 8    | 16   |
| Dynamic (s)          | 1.93 | 2.32 | 3.55 | 6.91 |
| threadPool (s)       | 2.09 | 2.85 | 4.1  | 7.23 |

# Selecting the Partition/Task size

- Choosing the task size (or partition size in static distributions) is critical for achieving good parallel performance
- The right size balances workload distribution against overhead introduced by task scheduling and synchronization.
- **Too small**: tasks incur high overhead due to frequent synchronization and scheduling
- **Too large**: minimize overhead but risks poor load balancing leading to idle threads
- **Optimal task size** is a trade-off between minimizing overhead and maximizing workload balancing



# Choosing a Workload Distribution Strategy

| Workload characteristics                                     | Preferred strategy     | Why                                                                    |
|--------------------------------------------------------------|------------------------|------------------------------------------------------------------------|
| Regular cost per task<br>Known number of tasks               | Static block           | Best locality, minimal scheduling overhead                             |
| Mild irregularity<br>Task cost varies moderately             | Cyclic or block-cyclic | Better balance than block distribution, still low overhead             |
| Strong irregularity<br>Unknown or data-dependent task cost   | Dynamic assignment     | Adapts at runtime, avoid idle Workers                                  |
| Very fine-grain irregular task<br>Large number of small task | Work-stealing          | Better scalability than work-sharing, less contention, better locality |

- Always tune chunk size: too small increases overhead, too large increases imbalance
- Use dynamic scheduling only when workload imbalance dominates execution time

# Suggested Readings

---

- Chapter 4 of the «Parallel Programming Concept and Practice» book
  - Additional readings:
    - «Scalable Load Distribution and Load Balancing for Dynamic Parallel Programs» by E. Berger and J.C. Browne
- Available here: <https://people.cs.umass.edu/~emery/pubs/wcbc-99-beb.pdf>